COMP 530 Assignment 6: Relational Operators

1. The Task

Your task is to provide implementations for four of the five relational operators in `Main/RelOps` (the fifth, `ScanJoin`, is provided for you, as a template).

The interfaces are in the headers in `Main/RelOps/headers`, with long comments that describe the interface. The operator headers are a bit intricate, but not that different from one another, and the `ScanJoin` code is your model for all of them. In particular, every operator takes its predicates and its projections as strings in MyDB's prefix notation (for example: `"&& (== ([l_suppkey], [r_suppkey]), > ([l_acctbal], double[1000.0]))"`), compiles this code into `func` objects using the `MyDB_Record` class, and evaluates each record that is scanned using the `func` object. Read `ScanJoin.cc` very carefully until you have an idea of how it works. Specifically, what does `[l_name]` refer to? How is a literal written into the prefix code? How does a predicate over two tables work? How is the record that is evaluated using such a two-table predicate built? How is a projection result written as an output attribute? What is `recordContentHasChanged ()` for? Understanding these is crucial because it will allow you to correctly instruct AI how to write the code for you.

Because the assignment pulls together many parts of the MyDB system (tables, the buffer manager, the schema infrastructure, the B+-tree, the sort, and the expression evaluation engine) it is a lot to take in. At this point, the code base is more than 8,000 lines.

2. How This Assignment Works

As in A4 and A5, AI is required: you will use AI to write the code and the tests. The assignment is organized in four stages, one operator for each stage.

What you write, you write yourself. The AI writes the code and the tests. The specification part of every stage prompt, your list of breaking changes, and `REFLECTION.md` must be produced by you, with no AI help of any kind. Using an AI to produce them is an honor code violation.

2.1 The Four Stages

For each stage, your opening prompt describes any details of the operator that are not already in the comments, specifies the answers to the design decisions (Section 3), describes how the prefix-notation predicates and projections are built and evaluated, suggests a set of tests and then asks for a final set of tests to approve before any code is written, and then finally asks for the code. Note that testing is a bit challenging here, as it is not often easy to know what the correct answer should be to a full relational operator. See Section 4 for a discussion of how you can handle this. The tests generated should go in `Main/RelOpTest/source/StudentQUnitStageN.cc`. Finally, once the tests have all been passed, you should describe to the AI a set of breaking changes for that operator, with at least two per stage and at least ten in all across the four stages (for example, "the final predicate is no longer applied" or "the last page of the input is never read"). Instruct the AI to apply one of them at a time, with all of your test suites run after each, with the breaking change undone before the next breaking change is applied. This is how you "test your tests." A change that no test is able to detect is an indication that your test suite is inadequate and needs to be fixed. We have a private list of ten mutations of our own that we will use to see whether they are caught by your test suite. Section 6 describes scoring.

The four stages you need to work on are:

1. **RegularSelection.** This simply scans all of the records in a file, applies a predicate to each, and then applies the projections to each record that passes. It is short and the place to get the prefix-notation machinery right.
2. **BPlusSelection.** This uses a B+-Tree to speed selection. The B+-tree's range iterator gives you the records with keys in [`low`, `high`]; the predicate and the projections are applied to the result. The B+-tree is my A4 solution; its interface is in `MyDB_BPlusTreeReaderWriter.h`.
3. **SortMergeJoin.** Sorts both inputs on their join keys into runs, merges them, and for every pair of records with equal keys, evaluates the final predicate over the combined record and projects. The sort code in `Sorting.h` is my A3 solution, and `buildItertorOverSortedRuns ()` in it does the run-building for you. Make sure to look it over to understand what it returns and what it does to the records you hand it.
4. **Aggregate.** Implements hash-based grouping and aggregation. Read Section 5 before writing the prompt. It has a set of useful questions past students asked and the answers I gave.

2.2 The Stage Prompts

Every stage starts with one opening prompt to an AI tool. Use one continuing conversation for the whole assignment if you can, so that the tool remembers the decisions it has already asked you about. If you start a new conversation for a stage, its opening prompt should somehow describe those prior decisions. The opening prompt has two parts. The first is a **preamble that you copy exactly** from `PROMPTS.md`. It identifies the course, assignment and stage, tells the tool to do only what the rest of the prompt asks, to use nothing but the handout, `FAQ.md`, the distributed code, and its general knowledge (no web search, no code repositories), and crucially to ask you when a decision is unclear rather than to choose itself. It names the files the tool may change in that stage, which includes that stage's operator `.cc` code, the test helpers, and that stage's test file. The operator headers, `ScanJoin.cc`, and everything under `DatabaseTable`, `Record`, `BufferMgr` and `Catalog` must be left alone.

After the part of the prompt that you copy, you include your specification, written by you.

When you submit your assignment, you must paste each opening prompt into `PROMPTS.md` in full, as you send it to the AI. *We also ask you to copy and paste any interactions with the AI as well as additional instructions that you give it* into the `PROMPTS.md` file. In A4 and A5 we asked you to export your conversation into `AILog/` and we will still ask you to do that, but some students reported that their history had been compacted and they lost the conversation. Thus this time we are asking you to copy/paste your interactions into the `PROMPTS.md` file, and that's what we will grade.

2.3 What Should My Prompt Look Like?

Section 2.4 of the A4 handout applies here. Your specification should attempt to describe the answer to each question you anticipate that the AI is going to run into. It should describe the cases to test, and ask for the tests to be listed and approved before any coding is done. The part specific to this assignment is the prefix notation. Had `ScanJoin` been assigned (and the code not been provided), a good specification for it would have contained something like this:

> Pin every regular page of the left table, and build an `unordered_map` from a hash value to a vector of `void *`. This contains the raw addresses of the left records on those pinned pages, obtained from the iterator's `getCurrentPointer ()`. (The pages must stay pinned for the rest of the run, because those addresses point into them.) The hash value of a left record is the XOR of the `hash ()` of each of the left-hand computations in `equalityChecks`. Compile each of those strings once, against one empty left record, with `compileComputation ()`, and compile the left

selection predicate the same way. Then scan the left table. Call `getCurrent ()` to load each record's contents into that record object; skip it if the predicate's result is not `toBool ()` true, otherwise hash it and record its address.

Then scan the right table. Compile the right-hand computations and the right selection predicate against one empty right record. For the final predicate and the projections, build a combined record: a new schema that stores the left table's attributes followed by the right table's, and a record over it created with `buildFrom (leftRec, rightRec)`, so that it shares the two input records' attribute values rather than copying them. Compile the final predicate and each projection against the combined record. For each right record that passes its predicate, compute its hash the same way and look it up. For each address in the bucket, `fromBinary ()` that address into the left record (the combined record now sees the pair), and if the final predicate is true, `set ()` each output attribute from the corresponding projection `func`, call `recordContentHasChanged ()`, and `append ()` to the output. The output table's schema is assumed to match the projections in number and type. No need to check it.

A Stage 1 prompt that says "implement RegularSelection so it sort of looks like ScanJoin" is not a specification!

3. Questions Your Prompts Have to Answer

Stage 1.

1. Where does the selection predicate come from?
2. What record is the predicate compiled against?
3. How is a projection string used to produce an attribute of an output record? Where does that output record come from?
4. How do we ensure that an output table's schema matches the projections?
5. When should `recordContentHasChanged ()` be called?

Stage 2.

6. Which of the B+-tree's iterators to use? How to supply `low` and `high`? Is the range inclusive at both ends? What happens with `low > high`, or when nothing is found in range?
7. The B+-tree is keyed on one attribute, but the predicate can mention any. What happens then?

Stage 3.

8. A predicate over two tables is compiled against what record? How is that record built, and how is it created from the two input records?
9. When several left records share a key and several right records share the same key, every possible pair has to be produced. Where are the left records of a group kept while the right side is consumed? What if they do not fit on a page?
10. `buildItertorOverSortedRuns ()` takes two records and a comparator. What does the iterator it returns do to those two records inside `advance ()`, and does it matter whether they are the same records your merge loop is holding? (It does, once there is more than one run. Work out why before the AI writes the loop.)
11. The selection predicates are applied to the inputs before sorting. Where, exactly?

Stage 4.

12. What is the schema of the record stored in the hash table, where does each attribute's type come from, and why is there one more attribute than the output has?

13. How is a group identified and hashed? What does the hash table map to, and how is a stored record found and updated in place?
14. How is each aggregate updated when a record arrives, and how does an average come out at the end?
15. What is the output when there is no `GROUP BY`? When no input record passes the predicate?

4. Testing

Testing is a bit tough for this assignment. It is not easy to know the right answer to a relational query over ten thousand suppliers. We have given you several ways to find out what the answer should be. Make sure to have the AI use them to produce test cases.

The oracle. `Build/oracle.py` loads the `.tbl` files into SQLite and runs any SQL you give it, printing the rows the way MyDB prints records (and sorted, with `--sorted`). Every operator call corresponds to an SQL query. So for any test, have the AI write the SQL, get the answer, and compare:

```
python3 oracle.py --sorted "SELECT name, acctbal FROM supplier WHERE nationkey = 1"
```

From a test, `oracle (sql)` in `TestHelpers.h` does the same and returns the rows. `dumpSorted (table)` dumps an output table the same way. `sameRows ()` compares the two as multisets, with a tolerance for doubles (a sum of many doubles can differ in the last digits depending on the order of the additions).

Cross-checks. Clearly, some tests can be constructed without an oracle. `ScanJoin` is provided and correct. Your `SortMergeJoin` must produce the same multiset of rows as a `ScanJoin` with the same arguments. `BPlusSelection` over the whole key range must agree with `RegularSelection` on the same predicate, and over a sub-range with `RegularSelection` with the range added to the predicate. For `Aggregate`, the group counts must sum to the total count. The number of groups is the number of distinct keys. AVG $\times$ CNT = SUM, and the no-`GROUP BY` aggregates can be checked against a plain loop over the input in the test.

Tiny tables with known answers. `likes.tbl`, `serves.tbl` and `freq.tbl` are the beer-drinker tables from lecture, with seven rows in all. It's easy to look at these and immediately know the answer. For example: who likes PBR and goes to Moe's? Homer. Five tests in `RelOpQUnit.cc` use them, one per operator. Beyond being easily checkable by hand, tiny tables are the only practical way to build the edge cases: an empty input, a predicate nothing satisfies, everything in one group, a join with no matches, duplicate keys on both sides.

The structure of `supplierBig`. This table has exactly 32 copies of each of the 10,000 rows of `supplier`, so a lot of facts regarding the output of operations on this table can be computed using simple arithmetic: a join of `supplier` with `supplierBig` on `suppkey` has exactly 32 rows per selected supplier, every group count on the key is 32, and so on.

The data. In `Build/` we have `supplier.tbl` (10,000 rows, `suppkey` 1 to 10,000), `supplierBig.tbl` (320,000), `nation.tbl` (25) and `region.tbl` (5) from TPC-H, and the three beer tables. `TestHelpers.h` has a function that builds the schema for each of these tables (`supplierSchema ()`, `nationSchema ()`, and so on). It also has `loadTable ()` and `loadBPlus ()`, which create a table from the schema and load it from a corresponding `.tbl` file. Each schema function takes a prefix that it puts in front of every attribute name, so that `supplierSchema ("l_")` and `supplierSchema ("r_")` give two copies of the supplier table whose attributes can be distinguished when the two are joined.

5. Questions and Answers From Previous Semesters

These are in `FAQ.md`, so your AI can read them.

Are we allowed to assume that the results of SUMs and CNTs are all integers and the results of AVG are doubles?

Basically, you don't worry about this! You should get the output schema (including the return types) from the output database table. The basic strategy is that you store records with schema identical to the output database table in your hash table, likely with one additional attribute that gives the total count (so you can do average). These records will have a number of aggregate attributes. For each of those attributes, you build up a string that specifies how to update the value stored in the aggregate attribute. For example, to handle a sum or an average, have some code like:

```
if (s.first == MyDB_AggType :: sum || s.first == MyDB_AggType :: avg) {
    aggComps.push_back (combinedRec->compileComputation ("+ (" + s.second + ", [
MyDB_AggAtt" + to_string (i) + "])"));
```

So the moral of the story is that you never really worry about the types. Everything sort of happens automatically, from the given output schema and the whole record/computation infrastructure.

I know that we can get `func` to compute an attribute value, but how can we get the attribute from `func`?

I don't think that you really want to get an attribute from a func. Example: say that you are trying to update an aggregate record stored in a hash table, and you've built a list of func objects that compute the new values that should be stored in an aggregate record. Then you might have code like:

```
for (auto &f : aggComps) {
    aggRec->getAtt (numGroups + i++)->set (f ());
}
```

In aggregation we are responsible to update an existing record in our own aggregation table when there is a match. I think we can take advantage of the `buildFrom ()` function which can concatenate two records' values, and then compile a function on it? But how do we do the computation over this concatenated record? Do you suggest any other easier way to update an existing record?

Here's (approximately) what I did. I first created a schema for my aggregate records, something like `<MyDB_agg_1, MyDB_agg_2, MyDB_agg_3, MyDB_group_1, MyDB_group_2, MyDB_cnt>`. I took the types of all of the agg attributes and group attributes from the schema embedded in `output`; the type of the last attribute is an integer. Then I create a combined record that consists of a record from the file to be aggregated, plus a record from this schema. Then, I create a function over the combined record, for each aggregate we are trying to compute. Let's say that the 2nd aggregate in `aggsToCompute` is a SUM aggregate. Then I would create an aggregate using something like the string `"+ (" + aggsToCompute[1].second + ", [MyDB_agg_2])"`. When we process a new input record, we do something like `aggRec->getAtt (1)->set (myFunction ());`. This will update the attribute in the aggregate record. Do this for every aggregate you are computing, and you are good to go!

As for the final step of the aggregate function, we need to iterate over the hash map and append everything into the output table. However, the records that I used for aggregation and the output table's records have different schemas. How do you suggest we

fix this? Or can we just call `output_record->fromBinary (address); output_tbl->append (output_record)` and assume the user is calling the aggregate function with the right schema?

You can assume that the user is calling the aggregate function with the correct schema. I actually have a check:

```
// make sure that the number of attributes is OK
if (output->getTable ()->getSchema ()->getAtts ().size () != aggsToCompute.size () +
groupings.size ()) {
    cout << "error, the output schema needs to have the same number of atts as (# of
aggs to compute + # groups).\n";
    return;
}
```

However, you need to be careful. My aggregate records don't have exactly the same schema as the output records, because I always add a count. Plus, not only that, you can't just blindly copy the values over (even if the schemas do match) because you might need to compute an average, which requires dividing by the count. In the end, I have something like:

```
// set the grouping atts
for (i = 0; i < numGroups; i++) {
    outRec->getAtt (i)->set (aggRec->getAtt (i));
}

// set the aggregate atts
for (auto a : finalAggComps) {
    outRec->getAtt (i++)->set (a ());
}
outRec->recordContentHasChanged ();
output->append (outRec);
```

6. Grading

Hidden tests	40%
Mutation score for your four test suites	25%
Your four prompts and your breaking changes	25%
`REFLECTION.md`	10%

Hidden tests. We run the tests in `RelOpQUnit.cc` plus others that are not public until after the turnin. The questions in Section 3 provide a good guide to what they cover.

Mutation score. We have ten broken versions of my operators, each with exactly one bug of the kind Section 2.1 describes. We compile your four `StudentQUnitStageN.cc` files and your `Main/RelOpTest/headers` against each one, with my operators and my base code otherwise. A broken version is caught if any of your suites reports a failure or crashes. Your score is the fraction caught. A suite that fails or crashes on my *correct* operators scores zero. The five distributed tests catch one of the ten.

Prompts. Your four opening prompts, from `PROMPTS.md`, are read against a rubric built from Sections 2.1, 2.3 and 3.

Reflection. `REFLECTION.md` is half a page.

7. Building, Testing, and Turning In

7.1 Building and Running

SCons option 1 builds `RelOpQUnit.cc` into `bin/relOpUnitTest`; options 2 through 5 build your Stage 1 through 4 suites into `bin/studentUnitTestStageN`. Add a `c` (`3c`) to build with clang++, as on Clear. Run everything from `Build`, where the `.tbl` files and `oracle.py` are. Each binary takes a test number to start from, and `N only` runs one test and stops. The `supplierBig` tests take a few seconds each to load; use `supplier` and the beer tables for most of your tests.

7.2 My Code, Your Code, and the Honor Code

As usual, I am giving out my solutions to earlier assignments in this one: the buffer manager, the record and page code, the sort, and the B+-tree. Simply stated, you are not allowed to distribute my code to anyone who is not taking the class right now... ever! Doing this constitutes a violation of the honor code. I'm particularly concerned about my code falling into the hands of students who might take the class in the future. Thus, a hard and fast prohibition on distributing my source code.

Your AI logs contain my code, so the same rule covers them. And a reminder of the rule from Section 2: the prompts, the breaking changes, and the reflection are written by you alone.

7.3 Turnin

Zip up your source code and turn it in on Canvas (make sure to archive into the zip format, and not some other archiving format; if you choose to use something else, we'll take off a few points!). Please name your archive `A6.zip`. Please do not change the original directory structure, except for adding new files. The root should be a directory called `A6`, with the two subdirectories `Build` and `Main`, plus `PROMPTS.md` with your four prompts filled in, `REFLECTION.md`, and `AILog`. You do not need to include the `.tbl` data files in your zip; they are large, and we have them.

And remember, to get any credit on A6, your code must compile and run on Clear. That way, we have a common environment for grading and we don't have to spend time getting your code to compile.

Finally, and this is important: include a `README` file in the root of your project with any important information, including the names of the one or two people who worked on the project, and which AI tools you used.

And also: if you work with a partner, only turn in one copy of your source. Otherwise, we'll possibly grade your submission twice, and end up getting quite annoyed.